

Làm tốt hơn nữa: từ lời giải bài *Parity* bàn về tối ưu hóa chương trình

Sun Linchun

Nguồn gốc: Let us do better: program optimization through the solution of Parity

IOI China candidate team papers / 2002 / Sun Linchun.pdf

1 Bài toán *Parity*

Ta có một dãy chỉ gồm các số 0 và 1. Một số thông tin được cho theo thứ tự đánh số: mỗi thông tin nói rằng trong một đoạn liên tiếp nào đó, ví dụ từ vị trí thứ ba đến vị trí thứ năm, số lượng phần tử 1 là lẻ hay chẵn. Những thông tin này có thể tự mâu thuẫn với nhau. Nhiệm vụ là tìm số hiệu lớn nhất k sao cho tồn tại một dãy 0, 1 thỏa tất cả các thông tin từ số hiệu 1 đến số hiệu k .

Độ dài dãy là L , với $1 \leq L \leq 10^9$. Số thông tin là N , với $1 \leq N \leq 5000$. Mỗi thông tin có dạng

$l \ r \text{ even}$ hoặc $l \ r \text{ odd}$,

nghĩa là số lượng 1 trong đoạn từ l đến r là chẵn hoặc lẻ.

Ví dụ.

Dữ liệu vào	Dữ liệu ra
10	
5	
1 2 even	
3 4 odd	3
5 6 even	
1 6 even	
7 10 odd	

Ba thông tin đầu có thể đồng thời đúng, nhưng khi thêm thông tin thứ tư thì không còn dãy nào thỏa mãn. Vì vậy đáp án là 3.

2 Khung xử lý chung

Tất cả các cách làm đều đi theo cùng một khung:

1. Đọc dữ liệu đầu vào.
2. Lần lượt đọc từng thông tin đoạn mới.
3. Kiểm tra thông tin mới có mâu thuẫn với các điều kiện đã biết hay không.
4. Nếu không mâu thuẫn, đưa thông tin đó vào hệ điều kiện đã biết.
5. Nếu mâu thuẫn, dừng lại và in số lượng thông tin đã xử lý hợp lệ.

Do đó điểm quyết định hiệu quả là cách biểu diễn “các điều kiện đã biết” và cách kiểm tra một thông tin mới.

3 Cách nghĩ ban đầu: thuật toán 1

Ý tưởng trực tiếp nhất là so sánh thông tin của đoạn hiện tại với từng đoạn đã biết, rồi phán đoán có mâu thuẫn hay không. Trước hết cần xét các quan hệ có thể có giữa hai đoạn.

Quan hệ	Dạng minh họa	Ý nghĩa
Tách rời	[đoạn 1] [đoạn 2]	Hai đoạn không có phần chung.
Trùng nhau	[đoạn 1] [đoạn 2]	Hai đầu mút đều giống nhau.
Giao nhau	[đoạn 1] [đoạn 2]	Có phần chung nhưng không đoạn nào chứa đoạn kia.
Chứa nhau	[đoạn 1] [đoạn 2]	Một đoạn nằm hoàn toàn trong đoạn kia.

Cách làm cụ thể như sau. Với đoạn hiện tại, ta lần lượt so sánh nó với các đoạn đã biết.

1. Nếu có một đoạn đã biết trùng hoàn toàn với nó, chỉ cần so sánh tính chẵn lẻ. Hai kết quả khác nhau tạo ra mâu thuẫn.
2. Nếu có đoạn đã biết cùng đầu trái hoặc cùng đầu phải, ta cắt bỏ phần chung giữa hai đoạn, đồng thời cập nhật chẵn lẻ của đoạn còn lại bằng phép xor của hai thông tin chẵn lẻ.
3. Đoạn mới sinh ra lại được đem so sánh với các đoạn đã biết. Quá trình lặp lại cho đến khi không còn đoạn đã biết nào có cùng đầu trái hoặc cùng đầu phải.
4. Cuối cùng, nếu chưa gặp mâu thuẫn, phần đoạn còn lại được đưa vào hàng đợi các đoạn đã biết.

Chẳng hạn, nếu hai thông tin có cùng đầu trái l ,

$$P[l, r] = x, \quad P[l, s] = y, \quad r < s,$$

thì thông tin trên phần còn lại là

$$P[r + 1, s] = x \oplus y.$$

Trường hợp cùng đầu phải được xử lý tương tự. Cách này đúng về mặt suy luận, nhưng mỗi thông tin mới có thể bị so sánh nhiều lần với cả hàng đợi điều kiện, nên hiệu quả chưa cao.

4 Phân tích sâu hơn: thuật toán 2

Cải tiến của thuật toán 2 là vẫn giữ một phần thông tin đoạn bị lặp, nhưng tập trung vào những đoạn có thể trực tiếp dẫn tới mâu thuẫn.

Khi đọc một đoạn mới và cần kiểm tra nó, nếu trong hàng đợi đã biết có những đoạn cùng đầu trái với đoạn hiện tại, ta chỉ giữ lại đoạn có đầu phải nhỏ hơn. Phần dài thêm của đoạn lớn hơn được xem như một đoạn mới, với tính chẵn lẻ được suy ra bằng phép xor như trên, rồi tiếp tục đem so sánh với các đoạn đã biết khác. Nếu không có đoạn nào cùng đầu trái với đoạn hiện tại, ta đưa đoạn hiện tại vào hàng đợi như đoạn đại diện cho đầu trái đó.

Cách nhìn này làm giảm số phép so sánh vô ích: với mỗi đầu trái, ta không cần giữ mọi đoạn như nhau về vai trò suy luận, mà chỉ cần giữ đoạn đại diện có khả năng sinh ra đoạn hiệu một cách trực tiếp.

5 Tối ưu cục bộ: thuật toán 3

Thuật toán 3 tối ưu khâu tìm kiếm bằng cách rời rạc hóa các đầu mút.

Cách đơn giản nhất là mở một mảng, dùng giá trị đầu trái làm chỉ số. Giá trị trong mảng là đầu phải của đoạn đại diện ứng với đầu trái đó; nếu không tồn tại đoạn như vậy thì ghi 0. Tuy nhiên L có thể tới 10^9 , nên mảng theo miền giá trị gốc quá lớn.

Ta cải tiến bằng rời rạc hóa. Đưa tất cả các đầu mút từng xuất hiện vào một mảng phụ, sắp xếp mảng này bằng quicksort, rồi đánh lại số hiệu mỗi đầu mút bằng vị trí tìm được qua tìm kiếm nhị phân. Khi đó mọi thao tác lưu trữ và tra cứu dùng chỉ số mới thay cho giá trị gốc. Cách này làm chi phí tìm đoạn đại diện nhỏ hơn nhiều so với việc quét tuyến tính hàng đợi.

6 Khai thác bản chất: thuật toán 4

Các thuật toán trên vẫn nhìn bài toán theo các đoạn. Thuật toán 4 đổi cách mô tả: thay vì chỉ ghi nhận chẵn lẻ của từng đoạn, ta chuyển sang chẵn lẻ tiền tố. Đây là bước khai thác bản chất của bài toán.

6.1 Hai cách mô tả

Với mô tả theo đoạn, đặt

$$P[i, j] = \begin{cases} \text{true}, & \text{từ vị trí } i \text{ đến vị trí } j \text{ có số lượng 1 lẻ,} \\ \text{false}, & \text{từ vị trí } i \text{ đến vị trí } j \text{ có số lượng 1 chẵn.} \end{cases}$$

Với mô tả theo tiền tố, đặt

$$\text{Parity}[i] = \begin{cases} \text{true}, & \text{từ vị trí 1 đến vị trí } i \text{ có số lượng 1 lẻ,} \\ \text{false}, & \text{từ vị trí 1 đến vị trí } i \text{ có số lượng 1 chẵn.} \end{cases}$$

Ta cũng cần $\text{Parity}[0]$, ứng với tiền tố rỗng.

Hai mô tả tương ứng qua công thức

$$P[i, j] = \text{Parity}[i - 1] \oplus \text{Parity}[j].$$

Vì vậy nếu $P[i, j] = \text{true}$ thì

$$\text{Parity}[i - 1] \oplus \text{Parity}[j] = \text{true},$$

còn nếu $P[i, j] = \text{false}$ thì

$$\text{Parity}[i - 1] \oplus \text{Parity}[j] = \text{false}.$$

6.2 Hai lớp tương đương

Xét tập mọi chỉ số của mảng tiền tố:

$$A = \{0, 1, 2, \dots, L\}.$$

Tập này được chia thành hai lớp tương đương theo giá trị $\text{Parity}[i]$: các chỉ số có giá trị true nằm trong một lớp, các chỉ số có giá trị false nằm trong lớp còn lại.

Theo công thức trên, giá trị của $P[i, j]$ quyết định $\text{Parity}[i - 1]$ và $\text{Parity}[j]$ có bằng nhau hay không. Nói cách khác, mỗi điều kiện trên đoạn $[i, j]$ trở thành điều kiện về việc hai phần tử $i - 1$ và j thuộc cùng một lớp tương đương hay thuộc hai lớp khác nhau.

6.3 Biểu diễn bằng hai tập quan hệ

Với mỗi chỉ số i , định nghĩa:

$\text{same}[i]$ là tập các phần tử đã biết cùng lớp với i ,
 $\text{opp}[i]$ là tập các phần tử đã biết khác lớp với i .

Nếu biết

$$\text{Parity}[j] = \text{Parity}[i],$$

thì $j \in \text{same}[i]$. Nếu biết

$$\text{Parity}[j] \neq \text{Parity}[i],$$

thì $j \in \text{opp}[i]$. Ban đầu

$$\text{same}[i] = \{i\}, \quad \text{opp}[i] = \emptyset.$$

Với mỗi thông tin mới, ta đổi đoạn $[l, r]$ thành cặp chỉ số tiền tố $(l - 1, r)$. Trong đoạn mã giả dưới đây, để gọn ký hiệu, cặp đang xét được viết là (i, j) .

process(i, j, x) :

nếu $x = \text{true}$ thì hai phần tử phải khác lớp;
 nếu $x = \text{false}$ thì hai phần tử phải cùng lớp;
 nếu yêu cầu mới mâu thuẫn với quan hệ đã biết thì dừng;
 ngược lại cập nhật các lớp quan hệ như sau:
 nếu $x = \text{true}$:
 $\text{merge}(\text{same}[i], \text{opp}[j]);$
 $\text{merge}(\text{same}[j], \text{opp}[i]);$
 nếu $x = \text{false}$:
 $\text{merge}(\text{same}[i], \text{same}[j]);$
 $\text{merge}(\text{opp}[i], \text{opp}[j]).$

Bản chất của hai nhánh cập nhật là: nếu i và j cùng lớp, thì các phần tử đối lập với i cũng đối lập với j ; nếu i và j khác lớp, thì tập cùng lớp với i phải nhập với tập đối lập của j , và ngược lại.

6.4 Hiện thực bằng hợp nhất tập rời nhau

Khi hiện thực, ta dùng cấu trúc hợp nhất tập rời nhau. Một cách quen thuộc là tạo hai đỉnh cho mỗi chỉ số u : đỉnh u biểu diễn lớp của u , còn đỉnh $\bar{u}$ biểu diễn lớp đối lập với u . Khi cần u và v cùng lớp, ta hợp nhất

$$u \leftrightarrow v, \quad \bar{u} \leftrightarrow \bar{v}.$$

Khi cần u và v khác lớp, ta hợp nhất

$$u \leftrightarrow \bar{v}, \quad v \leftrightarrow \bar{u}.$$

Mâu thuẫn xuất hiện nếu một phần tử bị buộc cùng lớp với chính phần tử đối lập của nó.

Với hợp nhất theo hạng và nén đường đi, độ phức tạp của cấu trúc này là

$$O(N\alpha(N)),$$

trong đó $\alpha(N)$ là hàm nghịch đảo của hàm Ackermann một biến; nó tăng chậm hơn nhiều so với $\log N$, nhưng về mặt lý thuyết không phải hằng số.

Sau khi phân chính của thuật toán đã giảm còn $O(N\alpha(N))$, khâu tra cứu chỉ số bằng tìm kiếm nhị phân $O(\log N)$ trở nên không cân xứng. Vì thế có thể dùng bảng băm để ánh xạ đầu mút gốc sang chỉ số rời rạc. Thời gian tra cứu khi đó là $O(1)$ kỳ vọng, tổng thời gian tra cứu là $O(N)$. Kết hợp hai phần, toàn bộ chương trình đạt độ phức tạp

$$O(N\alpha(N)).$$

7 So sánh thời gian chạy

Bảng sau ghi lại thời gian chạy của bốn cách hiện thực trên một số bộ dữ liệu. Các số liệu cho thấy tối ưu cục bộ có thể giúp đáng kể, nhưng bước đổi mô hình sang quan hệ tiền tố mới là cải thiện quyết định.

Dữ liệu	Thuật toán 1	Thuật toán 2	Thuật toán 3	Thuật toán 4
$N = 500$	0.00s	0.00s	0.00s	0.00s
$N = 5000$	3.41s	3.41s	0.22s	0.22s
$N = 2003$	1.87s	0.77s	0.38s	0.05s
$N = 4505$	35.93s	0.71s	0.27s	0.22s
$N = 5000$	10.77s	7.36s	7.14s	0.22s

8 Kết luận

Qua bài *Parity*, ta thấy vai trò quan trọng của việc tối ưu thuật toán và chương trình. Có hai quy luật chung đáng ghi nhớ.

1. Xuất phát từ chính bài toán, phân tích sâu để tìm bản chất của các ràng buộc.
2. Nhìn vào điểm yếu của thuật toán hiện tại rồi cải tiến đúng phần đang gây lãng phí.

Rốt cuộc, mọi tối ưu hiệu quả đều dựa trên sự hiểu biết sâu về bài toán và về cấu trúc dữ liệu. Chỉ khi nhận thức rõ bài toán, hiểu nó đầy đủ và sử dụng thành thạo các cấu trúc dữ liệu cần thiết, ta mới thiết kế và hiện thực được những thuật toán vừa hiệu quả vừa thực dụng.